

TECHIN 515 Lab 5 – Edge-Cloud Offloading

When ESP32 performs inference locally

```
connected.
Button pressed! Starting gesture capture...
Capture complete
First feature value: 1.25
Prediction: Z (98.44%)

--- Sending Prediction to Server ---
URL: http://10.18.215.67:8000/log
Payload: {"student_id":"marjyang","gesture":"Z","confidence":98.44}
HTTP Response code: 200
Server response: {
  "status": "logged"
}

--- End of Request ---
```

When ESP32 consults cloud for inference

```
quit: ctrl-c | nano: ctrl-c | nmap: ctrl-c | followed by ctrl-c
Button pressed! Starting gesture capture...
Capture complete
First feature value: 3.23
Prediction: Z (54.69%)
Low confidence - sending raw data to server...

--- Sending Raw Data to Server ---
URL: http://10.18.215.67:8000/predict
Payload: {"data": [3.229778, 8.14268, 5.303157, 2.554613, 8.51857, 4.939238, 2.410961, 8.487445, 4.096479, 2.063802, 8.659827, 3.062184, 2.11
, 1.697489, 1.458069, 8.944737, 1.242591, 1.328782, 9.172187, 1.233014, 1.517924, 9.141062, 1.893813, 1.09415, 8.714894, 0.452504, 0.35913, 8.4
5, 1.055843, 8.14268, -1.335965, -1.371878, 8.116343, -1.285686, -1.857901, 8.255207, -1.194707, -2.48997, 8.116343, -0.979228, -1.972822, 7.8
4417, -2.327164, 7.45315, -1.604115, -2.856283, 6.897695, -1.99437, -2.980781, 6.993463, -2.013524, -3.167529, 6.622362, -2.15957, -3.639187,
.480393, -4.407725, 6.318298, -2.499547, -4.572925, 5.947197, -2.552219, -4.979939, 5.643134, -2.590526, -5.437232, 5.57849, -2.470816, -5.57
48, -2.755726, -0.602119, 4.448427, -2.947262, -6.105214, 4.048595, -3.234566, -5.724536, 3.414132, -3.1795, -5.401319, 3.260903, -3.284845, -
3.10405, -3.292027, -1.836353, 1.884237, -3.040636, -0.42138, 1.810017, -2.698265, 0.840365, 1.572991, -2.44448, 2.142811, 0.76375, -2.726996,
66869, -2.236184, 5.329493, 0.876278, -1.862689, 6.83784, 0.215478, -1.513135, 8.664616, 0.395043, -0.134075, 9.272743, -0.038307, -0.888249,
.343147, -1.055843, 12.72758, -2.166753, -1.309628, 13.86961, -2.178724, -0.672771, 14.83926, -2.664747, -0.562637, 15.28458, -3.16274, -0.69
3, -3.979163, -0.897826, 16.83363, -4.007894, -0.701501, 17.7961, -4.474763, -0.746991, 18.97644, -4.869806, -0.703895, 19.93891, -5.351041, -
26608, -4.350265, -1.49877, 23.04659, -3.790021, -1.613692, 23.29319, -4.412514, -2.286463, 22.8766, -4.307169, -2.808398, 22.90293, -3.55539
2.55577, -2.128445, -3.361459, 22.73055, -1.194707, -2.753332, 23.08729, -0.244209, -1.714248, 22.31875, -1.084573, -0.907402, 19.9844, -3.41
509, 16.78096, -2.985569, -1.369483, 14.97094, -3.023877, -0.490811, 13.18008, -3.038242, 0.45011, 11.38204, -3.189077, 1.733402, 9.143456, -2
63, 4.984728, -0.64404, 3.136404, 2.985569, -0.11971, 3.562572, 1.106121, -0.071826, 3.931279, -0.361524, 0.469264, 4.357447, -1.278504, 1.414
, -3.536236, 2.006341, 4.946421, -4.175488, 2.355895, 4.434062, -4.824317, 2.688689, 4.010288, -4.936844, 2.985569, 3.768473, -5.152322, 3.253
-5.631162, 3.711013, 2.911349, -5.793968, 3.909731, 2.41575, -6.035782, 4.302381, 2.260126, -6.069301, 4.546589, 2.181118, -6.110003, 4.79079
5.829881, 5.523423, 1.687912, -5.638345, 5.949591, 1.472434, -5.408502, 6.349423, 1.288081, -5.125986, 6.675035, 1.170765, -5.030218, 7.06289
HTTP Response code: 200
Server response: {
  "confidence": 67.25149750709534,
  "gesture": "0"
}

Server Inference Result:
Gesture: 0
Confidence: 67.25%

--- Sending Prediction to Server ---
URL: http://10.18.215.67:8000/log
Payload: {"student_id":"marjyang","gesture":"0","confidence":67.25}
HTTP Response code: 200
Server response: {
  "status": "logged"
}

--- End of Request ---
```

Server side

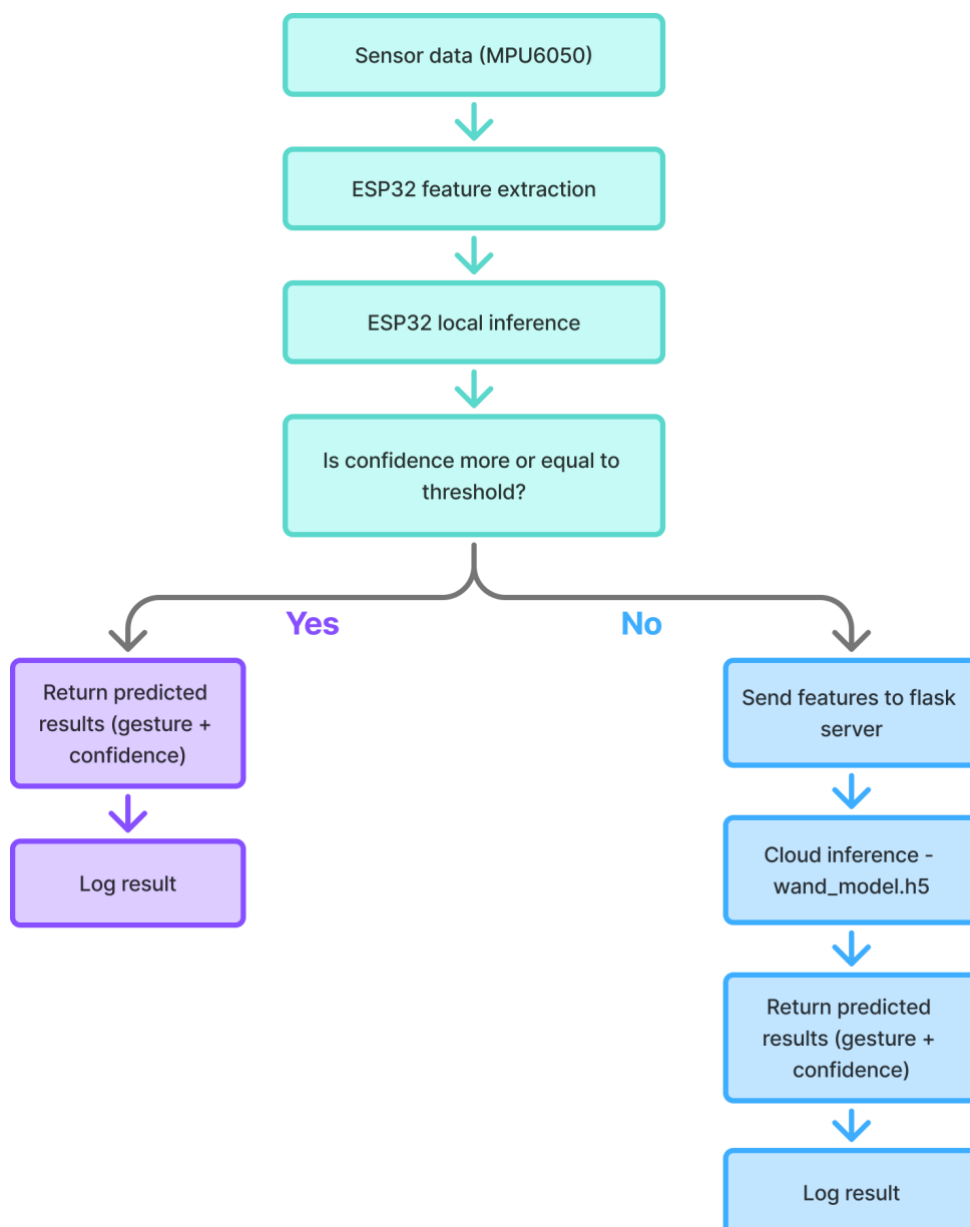
```
Root@cloud:~#
Starting Flask app...
* Debugger is active!
* Debugger PIN: 909-498-313
1/1 _____ 0s 38ms/step
10.18.96.118 - - [05/Jun/2025 14:48:04] "POST /predict HTTP/1.1" 200 -
📌 Logged prediction: {'student_id': 'marjyang', 'gesture': '0', 'confidence': 67.25}
10.18.96.118 - - [05/Jun/2025 14:48:04] "POST /log HTTP/1.1" 200 -
10.18.215.67 - - [05/Jun/2025 14:48:40] "GET / HTTP/1.1" 200 -
📌 Logged prediction: {'student_id': 'marjyang', 'gesture': 'Z', 'confidence': 98.44}
10.18.96.118 - - [05/Jun/2025 15:28:17] "POST /log HTTP/1.1" 200 -
```

Discussion

1) Is server's confidence always higher than wand's confidence from your observations? What is your hypothetical reason for the observation?

From my observations, yes – when the wand's confidence is low and the system falls back to the server, the server's confidence tends to be higher. The server is able to do this because it is larger, has more capacity (more layers), and regularization which helps prevent overfitting. The server also likely trains on more complete data and it has access to more compute power meaning it can maintain higher accuracy and confidence. On the other hand, the wand model is lightweight and optimized for real-time performance on constrained hardware which means it might produce lower confidence on varied data.

2) Sketch the data flow of this lab



3) Our approach is edge-first, fallback-to-server when uncertain. Analyze pros and cons of this approach from the following aspects: reliance on connectivity, latency, prediction consistency, data privacy.

	Pros	Cons
Reliance on connectivity	The system can function without Wi-Fi or cloud connection.	Unreliable when the confidence is low - a poor connection will block server access, leading to delayed or failed inference.
Latency	Local inference is much faster – with real-time responsiveness.	Server fallback leads to extra latency because of data transmission and server processing. This would not be in real-time and can be a bigger disadvantage in time-sensitive use cases.
Prediction consistency	Cloud/server models can be updated more frequently with more training data – improving over time	There might be some discrepancy between the edge and server model predictions. This might especially be the case if only the server model is retrained.
Data privacy	Local inference means that user data is on-device – enhancing privacy.	Sending raw sensor data to the cloud could lead to potential risks of data exposure. This is especially a disadvantage if the use case is more sensitive.

4) Name a strategy to mitigate at least one limitation named in question 3.

Using edge-cloud collaborative inference could address the latency and prediction consistency limitations. This approach would allow partial computation on the edge and offloads only what's needed to the cloud for final prediction, reducing latency compared to full data transmission and ensuring the edge and cloud stay in sync.